Laboratorio 06

DOCENTE	CARRERA	CURSO
DSc. Edgar Sarmiento Calisaya	Escuela Profesional de Ciencia de la Computación	Ingeniería de Software II

LABORATORIO	TEMA	DURACIÓN
06	Pruebas Funcionales	2 horas

1. Competencias del curso

- Ejecuta pruebas estáticas y dinámicas de componentes de software mediante herramientas y técnicas de caja blanca y caja negra, en colaboración con los clientes.
- Trabaja individualmente y en equipo, de manera ética y con compromiso, para el desarrollo de soluciones.

2. Competencias de la práctica

- Automatización de pruebas funcionales.

3. Equipos y materiales

- Computadora
- Internet
- Material específico del curso descrito en el aula virtual:

4. Objetivo

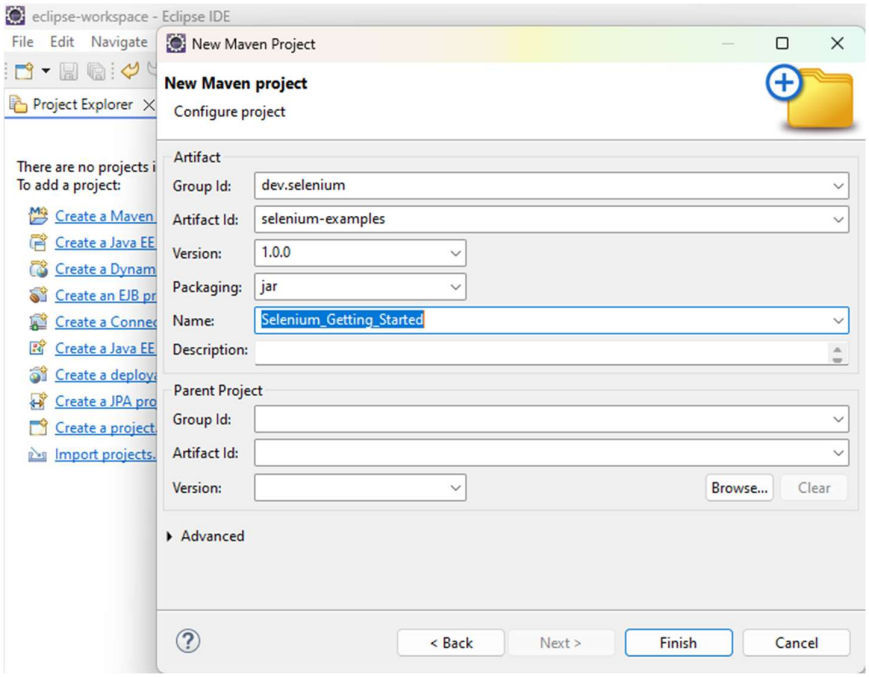
- Automatizar las pruebas funcionales utilizando los frameworks **Selenium** y **xUnit,** maximizando la cobertura de pruebas.

5. Prerrequisito

- Java 17 (También pueden implementarlos en Python, C# o JS)
- IDE:
  - Java: https://www.eclipse.org/downloads/packages/ - zip o tar.gz
- Selenium Web Driver: https://www.selenium.dev/documentation/webdriver/
- Proyecto Java + Maven: https://github.com/springframeworkguru/springbootwebapp

6. Actividades

a) Crear un proyecto de pruebas funcionales (de acuerdo al lenguaje de programación elegido). Los scripts presentados en este documento están basados en Java. Por ejemplo:



b) Importar **Selenium Web Driver** en su proyecto vía su gestor de dependencias o sistema de construcción automática: https://www.selenium.dev/documentation/webdriver/getting_started/install_library/ Por ejemplo:

```
<properties>
   <maven.compiler.source>17</maven.compiler.source>
   <maven.compiler.target>17</maven.compiler.target>
   <selenium.version>4.26.0</selenium.version>
</properties>

<dependencies>
   <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>${selenium.version}</version>
   </dependency>
</dependencies>

<build>
 <pluginManagement>
  <plugins>
   <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.8.1</version>
   </plugin>
  </plugins>
 </pluginManagement>
</build>
```

c) Ejecutar el siguiente **proyecto de prueba**:
https://github.com/springframeworkguru/springbootwebapp

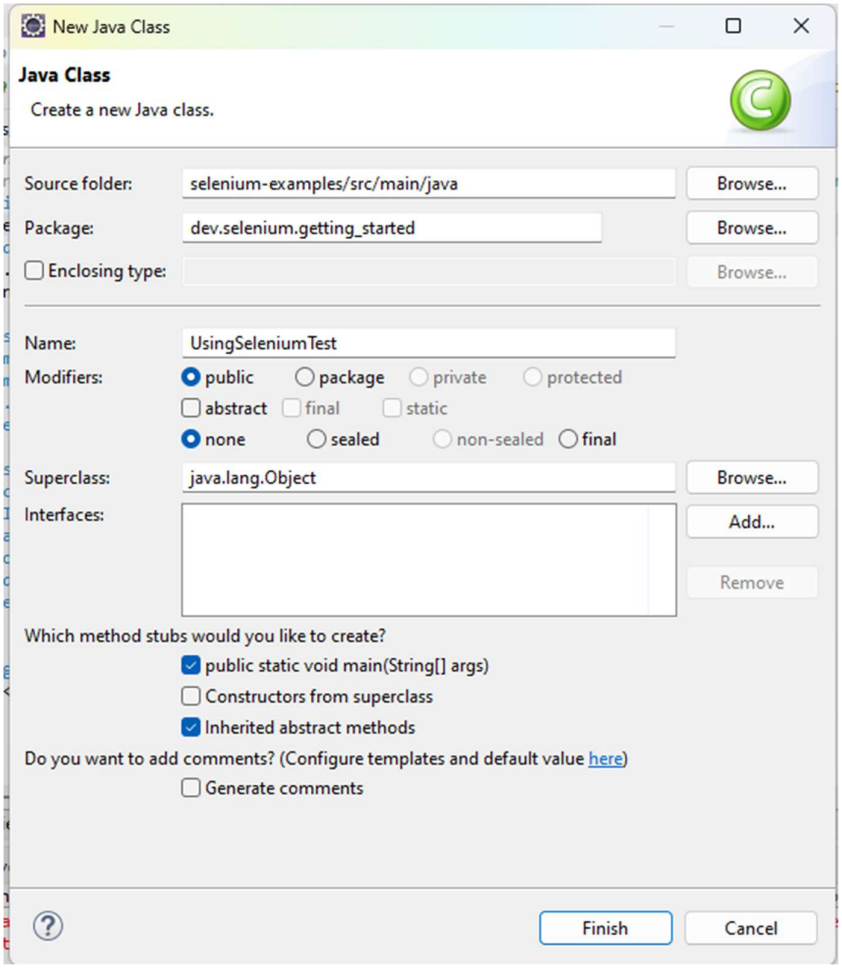
- Compilar el proyecto (vía su IDE) o: `mvn clean package`
- Ejecutar el proyecto: `java -jar target/spring-boot-web-0.0.1-SNAPSHOT.jar --server.port=8083`
- Verificar su funcionamiento: http://localhost:8083

d) Implementar los casos de prueba para la funcionalidad "***Create Product***" del **proyecto de prueba**:

- Diseñar los casos de prueba utilizando alguna estrategia basada en caja negra (p.ej., clases de equivalencia, valores limite, tablas de decisión, casos de uso, BDD, ...)

Objeto de Prueba / Funcionalidad	**Escenario/Acciones**	**Valores de Prueba**	**Resultado Esperado**

- Implementar los casos de prueba (1 método por caso de prueba) por medio de scripts de prueba utilizando los elementos proporcionados por Selenium: https://www.selenium.dev/documentation/webdriver/getting_started/first_script/. Por ejemplo:



```java
package dev.selenium.getting_started;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class UsingSeleniumTest {



        public static void main() {
                WebDriver driver;
                driver = new ChromeDriver();

                driver.manage().timeouts().implicitlyWait(Duration.ofMillis(500));
                driver.get("https://www.selenium.dev/selenium/web/web-form.html");

                String title = driver.getTitle();
                System.out.println("Web form: " + title);

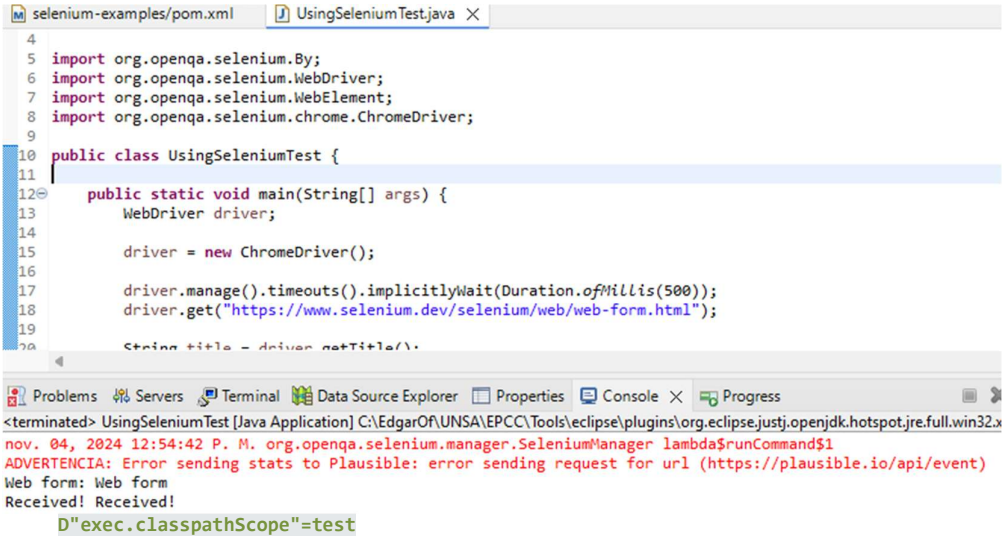
                WebElement textBox = driver.findElement(By.name("my-text"));
                WebElement submitButton = driver.findElement(By.cssSelector("button"));

                textBox.sendKeys("Selenium");
                submitButton.click();

                WebElement message = driver.findElement(By.id("message"));
                String value = message.getText();
                System.out.println("Received! " + value);

                driver.quit();
        }
}
```

- Ejecutar los casos de prueba vía su IDE o: `mvn exec:java -D"exec.mainClass"="dev.selenium.getting_started.UsingSeleniumTest" -`



`D"exec.classpathScope"=test`

e) Organizar y ejecutar los casos de prueba por **xUnit**: https://www.selenium.dev/documentation/webdriver/getting_started/using_selenium/

- Importar **xUnit** (p.ejm., JUnit) en su proyecto vía su gestor de dependencias o sistema de construcción automática. Por ejemplo, actualizar pom.xml:

```xml
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>5.11.3</version>
    </dependency>
```

```java
package dev.selenium.getting_started;

import static org.junit.jupiter.api.Assertions.assertEquals;

import java.time.Duration;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class UsingSeleniumXUnitTest {

	WebDriver driver;

	@BeforeEach
	public void setup() {
		driver = new ChromeDriver();
	}

	@Test
	public void eightComponents() {

		driver.manage().timeouts().implicitlyWait(Duration.ofMillis(500));
		driver.get("https://www.selenium.dev/selenium/web/web-form.html");

		String title = driver.getTitle();
		assertEquals("Web form", title);

		WebElement textBox = driver.findElement(By.name("my-text"));
		WebElement submitButton = driver.findElement(By.cssSelector("button"));

		textBox.sendKeys("Selenium");
		submitButton.click();

		WebElement message = driver.findElement(By.id("message"));
		String value = message.getText();
		assertEquals("Received!", value);

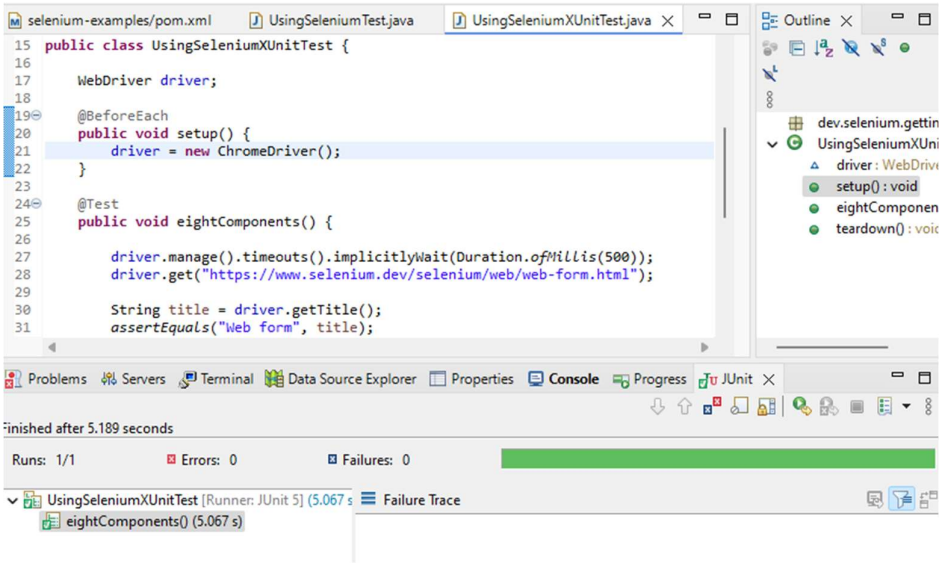
	}

	@AfterEach
	public void teardown() {
		driver.quit();
	}

}
```

- Implementar los casos de prueba (1 método por caso de prueba) utilizando método o macros xUnit (***Test Suites, Test Cases, Assertions, SetUp y TearDown***). Por ejemplo:

- Ejecutar los casos de prueba vía su IDE y su plugin/extension xUnit, o: `mvn exec:java -D"exec.mainClass"="dev.selenium.getting_started.UsingSeleniumXUnitTest" -D"exec.classpathScope"=test`



7. Entregables

- Casos de prueba identificados (formato tabular) para el "**proyecto de prueba**"

Objeto de Prueba / Funcionalidad	**Escenario/Acciones**	**Valores de Prueba**	**Resultado Esperado**

- Proyecto de automatización de pruebas funcionales para el "**proyecto de prueba**"
- Reporte de ejecución de pruebas: xUnit

8. Conceptos y Documentación

- Material sobre Blackbox Testing
- Selenium Web Driver: https://www.selenium.dev/documentation/webdriver/